

LT3 Liability Trading: From Full-Depth Order-Book Research to a Hardened Competition Engine

Nathaniel Holbrook Coulter*

Columbia University — Mathematics of Finance

September 2026

Abstract

This report documents the development, validation, and in-class deployment of my automated strategy for the Rotman Interactive Trader Liability Trading 3 (LT3) case. My central premise was that LT3 should be treated as a liquidity-pricing and execution problem rather than as a directional trading game. The stock process is specified as a zero-mean random walk, while the case explicitly asks the trader to evaluate block liquidity, accept only appropriate tender spreads, and then unwind accepted client inventory. I therefore built a sequence of research and execution engines that priced the *entire* tender quantity against visible depth, incorporated fees, risk limits, a safety reserve, and later full-depth liquidity imbalance, and separated ex-ante selection edge from subsequent random market movement.

The project progressed from a simple full-book VWAP engine (V1), to a persistent 30-tick/full-depth research database (V2), to live execution calibration (V3), to a deliberately failed latency experiment that exposed a simulator booking boundary (V4), to a gated low-latency execution architecture (V4.1), to selection optimization (V5), and finally to a frozen booking-risk model tested only in shadow mode (V6). The model added statistical signal but did not improve fresh economic decisions, so it was not promoted. The final competition build therefore retained the simpler V5 policy and hardened the operational layer instead.

During the seven scripted class heats, the engine evaluated 35 tenders, accepted 8, executed 41/41 hedge orders successfully, traded 553,000 shares, incurred zero fines, used zero repair orders, and ended flat after every accepted tender. All 8 accepted tenders were profitable. Ex-ante arrival-book profit on the selected tenders was \$57,119.23; realized security-level P&L was \$70,184.88. The two best scripted heats realized \$20,148.12 and \$18,426.00. More importantly, the competition data demonstrated why full-block pricing mattered: 30 of 35 tenders looked profitable at the best quote after fees, but only 14 of 35 remained positive when the whole block was swept through visible depth. Among the 23 tenders with enough visible depth to liquidate the full block, blindly taking all of them had contemporaneous after-fee economics of approximately -\$75,623, while the strategy-selected subset had +\$57,119 of observable arrival-time edge.

1 Case framing and research philosophy

I approached LT3 as a miniature liability-trading problem. The case supplies five institutional block orders over a five-minute period, with portfolio net and gross limits, per-share transaction fees, ticker-specific maximum order sizes, and an explicit restriction against front-running or unrelated speculative trading. The case documentation also states that the simulated stock follows a zero-mean random walk and explicitly distinguishes forecasting liquidity from forecasting exogenous price direction. That distinction became the organizing principle of the project.

*GitHub Repository: github.com/Nathaniel-Coulter/Columbia-Rotman-Trading-Competition

My goal was not to maximize the variance of the leaderboard outcome. It was to maximize *observable ex-ante edge* subject to liquidity, execution, and risk constraints. If an institutional tender did not have a defensible positive liquidation edge using information available at the moment of decision, I preferred to decline it rather than hope the random walk moved in my favor later. This meant accepting that I might underperform a more aggressive trader in a short realization even if the process itself was better controlled.

1.1 Full-block edge, not quoted spread

For a tender in which I buy shares from the institution, I become long and must sell into bids. For a tender in which I sell shares to the institution, I become short and must buy back through asks. If Q is the tender quantity, P_T is the tender price, and $VWAP_{bid}(Q)$ or $VWAP_{ask}(Q)$ is the price of sweeping the required side of the visible book for the entire quantity, then the raw liquidation edge was

$$E_{raw} = \begin{cases} VWAP_{bid}(Q) - P_T, & \text{BUY tender,} \\ P_T - VWAP_{ask}(Q), & \text{SELL tender.} \end{cases} \quad (1)$$

The fee-adjusted edge was

$$E_{fee} = E_{raw} - f_{market} - f_{tender}, \quad (2)$$

where the research ultimately established $f_{market} = \$0.02/\text{share}$ and $f_{tender} = 0$ in the active RIT build. I then reserved a safety buffer $b = \$0.03/\text{share}$ for adverse movement between observation and executable hedge time:

$$E_{decision} = E_{fee} - b. \quad (3)$$

The original conservative policy required $E_{decision} \geq \$0.03/\text{share}$, which implied roughly a 6-cent after-fee arrival hurdle. V5 later reduced the residual requirement to 2 cents, creating a 5-cent core hurdle while preserving the same 3-cent safety reserve and the \$750 minimum modeled profit requirement.

This distinction between quoted spread and executable block edge was not cosmetic. It became the most important empirical result from competition day (Figure 1).

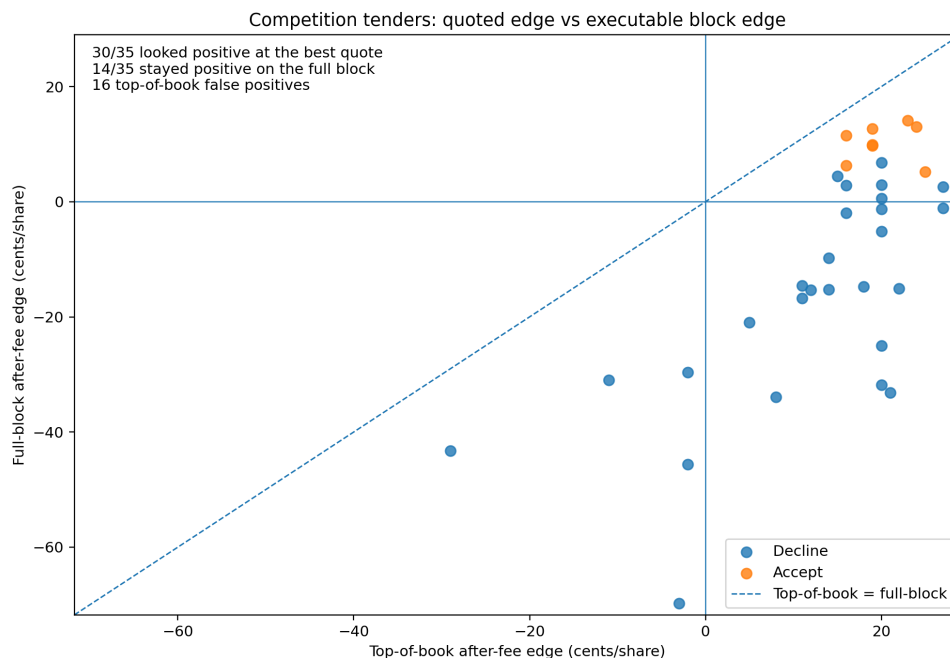


Figure 1: Competition tenders: best-quote after-fee edge versus full-block after-fee edge. A favorable best quote was often a false positive for a 65–75k-share block.

2 Chronological alpha research and feature engineering

2.1 V1: establish the economically correct baseline

The first engine was deliberately simple. It polled active tenders, fetched the live book, swept the required side until either the tender quantity was fully covered or visible liquidity was exhausted, checked projected net/gross limits, applied the fee and safety assumptions, and then scored the tender. If live mode were enabled, accepted inventory would be flattened with market orders split by the case maximum order sizes: 25,000 shares for CRZY and 10,000 for TAME.

The important V1 design choice was conceptual rather than computational: I refused to use the best bid or ask as if the entire block could trade there. The engine priced the whole unwind curve. It also separated tender acceptance from post-acceptance hedging so that it would never intentionally front-run a pending client order.

V1 used the initial conservative parameters:

- market hedge fee: 2 cents/share;
- tender fee assumption: zero, to be empirically validated;
- safety reserve: 3 cents/share;
- residual required edge: 3 cents/share;
- minimum modeled profit: \$750;
- hard flatten backstop before the end of the 300-tick period.

At this stage I had a rule, not yet an empirical research process.

2.2 V2: turn the simulator into a dataset

V2 was the first substantial research upgrade. I kept live trading off and changed the objective from “make a decision” to “record enough information that later decisions can be tested.” The engine created a persistent SQLite database, detected separate five-minute heats across script restarts, and stored a

tender master table plus 30 follow-up snapshots after every tender. It also stored up to 60 levels per side of the order book during those follow-ups.

For each tender and follow-up I recorded, among other variables:

- bid, ask, midpoint, and spread;
- full-block liquidation VWAP and worst consumed price;
- 25/50/75/100% VWAP curve;
- visible coverage and unwind depth ratio;
- top-5, top-10, and full-depth imbalance;
- breakeven, buffered, and threshold-visible quantity;
- counterfactual after-fee liquidation P&L;
- projected portfolio net/gross risk.

This converted LT3 from a sequence of isolated five-minute games into a reproducible microstructure dataset. The V2 run contained 76 tender observations, with 75 clean one-tick follow-ups. A useful pattern emerged: favorable *full-depth* imbalance had a statistically meaningful relationship with subsequent short-horizon edge stability, while top-5 and top-10 imbalance did not. Using the one-tick follow-up as the short-horizon proxy, full-depth favorable imbalance had Spearman $\rho \approx 0.315$ with edge change ($p \approx 0.006$); top-5 and top-10 measures were not significant.

I interpreted this carefully. It was not a directional price forecast. It suggested that the shape of deep visible liquidity contained information about whether the liquidity required for a block unwind was likely to persist over the next second or so. That was directly relevant to liability trading.

2.3 V3: calibrate real execution rather than paper execution

Paper mode could not answer whether the predicted sweep VWAP would actually match RIT fills, how fees were applied, or where the latency lived. V3 therefore kept the V2 research database but added live execution and detailed telemetry.

V3 recorded accepted-tender timing, per-order submit latency, position/fill observation latency, predicted versus realized slice VWAP, actual flattening quantity, P&L snapshots, and counterfactual thresholds. Across 30 observed tenders, V3 accepted 12 and submitted 55 hedge orders, covering 852,000 tender shares. All 12 accepted positions ended flat.

The arrival-book model predicted \$104,009 of after-fee profit across those 12 trades; the realized tender/hedge economics implied \$111,324, approximately \$7,315 better than the untouched arrival estimate. Mean aggregate execution error was approximately -0.865 cents/share (negative means better than predicted), median -0.590 cents/share, and the worst adverse aggregate miss was about +2.374 cents/share.

More importantly, V3 decomposed latency. Tender-accept HTTP responses averaged about 36.9 ms, but the accepted tender position did not appear in the portfolio state for an average of about 855.8 ms (range roughly 775–933 ms). Once individual hedge slices were sent, fill/position observation averaged about 171.8 ms per slice. The actual market-order execution itself was very faithful to the book: 54 of 55 slice VWAPs exactly matched the immediately predicted slice VWAP, with the remaining slice improving by roughly 1.55 cents.

That led to the first major engineering conclusion: most of the wall-clock execution time was not being spent crossing the book; it was being spent waiting for simulator state to update between slices.

2.4 V4: the useful failed experiment

V4 attacked that latency aggressively. It increased the main tender poll frequency, read the case-advertised API order rate dynamically (200 orders/second in the demo build), removed the old hard-coded 105 ms inter-order pacing, and pipelined the exact tender quantity in max-size market-order slices. It also

made unsafe POST requests non-retriable after ambiguous transport failures, because blindly retrying an order or tender acceptance can duplicate an action.

The key V4 assumption was that a successful tender-accept HTTP response was sufficient authorization to begin the hedge immediately. That made the engine fast: accept-to-first-order fell to roughly 0.1 ms after the response, CRZY three-slice bursts were around 110–140 ms, and TAME multi-slice bursts were around 270 ms.

The assumption was wrong for the simulator’s internal liability/fine accounting.

In the final diagnostic V4 heat, the three accepted tenders generated \$30,512 of economic after-fee profit, but RIT assessed \$81,400 of fines, producing a displayed P&L of -\$50,888. Position telemetry showed the mechanism: the hedge orders became visible before the accepted tender inventory had been booked into the portfolio. For a BUY tender, the position monitor briefly showed the hedge-created short position before the tender-created long position appeared. Economically I was unwinding an accepted client block, but the simulator’s fine engine temporarily saw speculative inventory.

This was a valuable failure because it separated two concepts that had looked identical from the HTTP interface: *tender accepted* and *tender liability booked*. It also exposed a secondary logging problem: very fast CRZY bursts could finish before security-level realized P&L propagated, and the asynchronous trader scoreboard could be stale. From this point forward, security-level realized P&L became the authoritative accounting source and account-level scoreboard fields were treated as secondary diagnostics.

2.5 V4.1: identify the simulator’s hard synchronization floor

V4.1 preserved the V4 pipeline but inserted one mandatory gate:

`accept response → exact tender position booked → refresh book → pipeline hedge.`

This fixed the fines problem without returning to V3’s slow per-slice position waits. The result was the most important execution-calibration dataset of the project: 163 tenders, 76 accepted executions, 372 hedge orders, and 5.327 million tender shares.

All 76 accepted tenders were profitable, all 76 ended flat, all 372 hedge orders were transacted, no repair orders were required, no API rate-limit hits occurred, and no new fines were recorded. Realized security-level P&L was 709,829 *versus* 699,356 predicted from the untouched arrival books.

The simulator booking delay was extremely stable. Mean accept-to-booked latency was 950.1 ms; the median was 948.7 ms; the 5th–95th percentile range was approximately 927.4–976.9 ms; and the maximum observed value was 986.5 ms. This revealed a practical hard latency floor of roughly one second that was imposed by simulator state synchronization rather than by Python. Once the position was booked, V4.1 needed only about 3.95 ms on average to begin the hedge. Average submission bursts were about 123.5 ms for CRZY and 282.7 ms for TAME; average booked-to-target-position time was about 258 ms and 438 ms respectively.

That changed the optimization target. Post-booking execution was no longer the main uncertainty. The main uncertainty was what happened to executable liquidity during the unavoidable booking interval.

The full-depth imbalance result also survived out of sample. On the 76 exact V4.1 booking observations, favorable full-depth imbalance had Spearman $\rho \approx 0.262$ with booking-period edge change ($p \approx 0.022$). Combining the independent V2 one-tick sample with the V4.1 exact-booking sample produced 151 observations with $\rho \approx 0.292$ ($p \approx 2.7 \times 10^{-4}$). The top-5 and top-10 variants remained substantially weaker.

2.6 V5: stop optimizing execution and optimize selection

By V5 I considered the execution problem largely solved. The engine therefore left the tender-booking gate and pipelined market hedge unchanged and moved the experiment to tender selection.

The effective core threshold was reduced from approximately 6 cents after fees to 5 cents after fees by keeping the 3-cent safety reserve and reducing the required residual edge from 3 cents to 2 cents. I also added an explicitly bounded marginal regime for 4–5 cent opportunities: such a tender could only be accepted if the entire block was visibly covered, all normal risk/profit constraints passed, and favorable full-depth imbalance was at least +0.05. The engine simultaneously recorded a dense edge/imbalance counterfactual grid so that threshold choices could be analyzed without rewriting history.

V5 observed 34 tenders and executed 25, using 121 hedge orders for 1.738 million shares. Realized P&L was \$187,118; 24 of 25 accepted tenders were profitable; there were zero fines, repairs, and rate-limit hits. The four trades that were accepted by the new 5-cent core rule but would have been excluded by the old 6-cent rule contributed +12,533 in realized P&L on the same path. One of the four lost 325 after a 5.15-cent arrival edge deteriorated by approximately 5.64 cents during booking, but the other three more than compensated. That was the clearest evidence that the 5-cent core threshold improved expected opportunity capture even though the 3-cent safety buffer was not a hard guarantee against every one-second adverse move.

V5 also corrected the fee-model diagnostic so that security-level realized P&L could no longer be overwritten by a stale account-level scoreboard delta. From V5 onward, the one-sided 2-cent market-hedge fee model matched security-level realized P&L to numerical precision.

2.7 V6: test whether a small model actually beats the rule

I did one final modeling experiment rather than continuing to hand-tune cent thresholds. V6 left the V5 policy in control of all live decisions and ran a frozen booking-risk model in shadow mode. The model was deliberately small. Its base features were:

$$X = [\text{arrival after-fee edge, favorable full-depth imbalance, depth ratio, spread, ticker, tender side}]. \quad (4)$$

An L2-regularized logistic model estimated the probability that a fully executable liquidation would remain profitable at booking. A robust Huber regression estimated booking-period edge change and included one interaction between arrival edge and favorable imbalance. The model was trained only on pre-V6 data and frozen before the V6 heats. Training contained 235 observations, 183 positive booking outcomes. Grouped out-of-fold calibration produced ROC AUC ≈ 0.987 , calibrated Brier score ≈ 0.0335 , and the robust regression had out-of-fold MAE of approximately 1.00 cent/share. The model gate required a calibrated probability of at least 0.90 and a positive downside-10 booking-edge estimate.

Fresh V6 testing produced 45 tender outcomes and 20 actual V5-controlled executions. On those 45 untouched outcomes, the frozen model still had ROC AUC ≈ 0.957 ; regression MAE was approximately 1.20 cents/share and predicted versus actual edge-change correlation was approximately 0.61. Statistically, the model was real.

Economically, it did not beat V5. The decisive observation was a 4.42-cent CRZY BUY with favorable full-depth imbalance of approximately +0.145. V5 accepted it under the marginal regime. V6 assigned a calibrated probability of about 0.884, just below its 0.90 gate, and therefore would have declined it. The trade’s booking edge improved to approximately 5.66 cents and it realized +\$3,791. The model did not avoid a separate loser to offset that missed profit. On the fresh path, V5 therefore beat the shadow policy by exactly \$3,791.

That was my stopping criterion. The additional model features contained information, but not enough incremental economic value to justify model risk in a short competition. I froze the alpha logic at V5 rather than promoting V6.

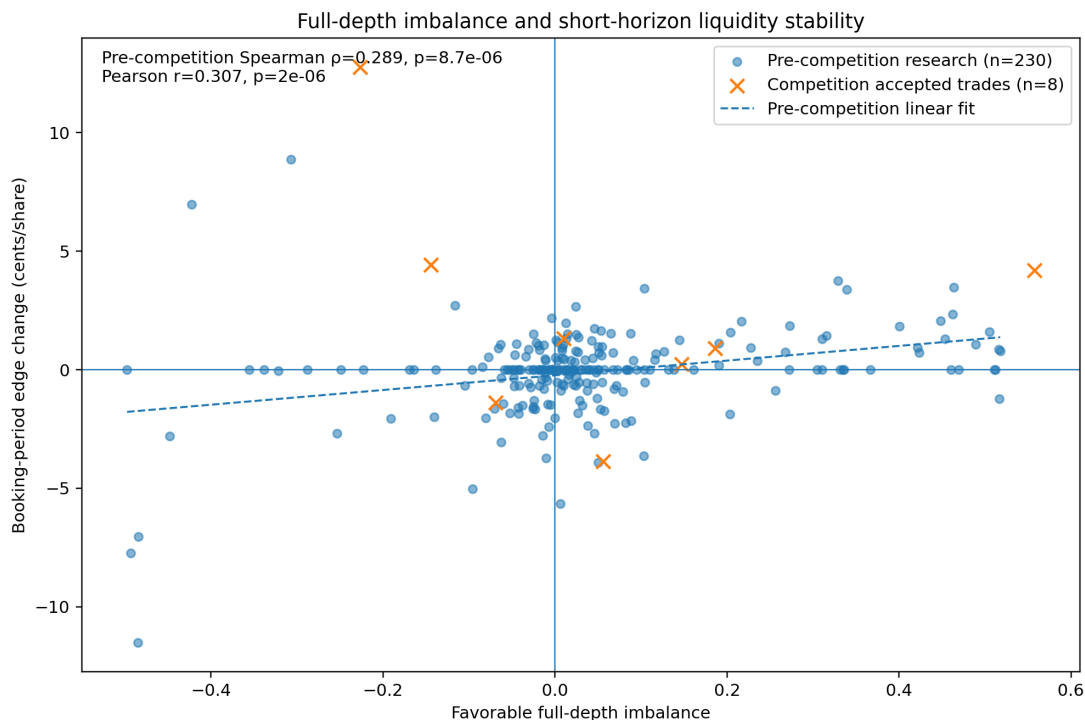


Figure 2: Full-depth imbalance retained a modest but repeatable relationship with short-horizon liquidity stability across the pre-competition research sample. The competition accepted trades are overlaid only for context; the class build intentionally did not record follow-up outcomes for declined tenders.

Across the pre-competition V2/V4.1/V5/V6 observations used in Figure 2, there were 230 clean short-horizon outcomes. Favorable full-depth imbalance had Spearman $\rho \approx 0.289$ ($p \approx 8.7 \times 10^{-6}$) and Pearson $r \approx 0.307$ ($p \approx 2.0 \times 10^{-6}$). I regard that as evidence of a real liquidity-stability signal, not as a precise price forecast. V6’s failure to improve economic decisions is the complementary result: a variable can be statistically predictive without justifying a more complicated live policy.

2.8 Competition build: harden the proven logic instead of adding alpha

The final `lt3_competition_engine.py` was not a new research strategy. It was a hardened V5 deployment. I retained the 5-cent core rule, the bounded 4–5 cent favorable-imbalance regime, the 3-cent safety reserve, full-block coverage requirement, risk limits, and tender-booking gate. I then reduced operational failure modes:

- read-only startup preflight against live case parameters;
- verification of CRZY/TAME fees, order-size limits, risk limits, API availability, and clean starting positions/orders;
- single-instance process lock;
- stale/near-expiry tender guard;
- safe retry policy for idempotent GET requests but no blind retry of ambiguous POST requests;
- fail-closed behavior for new fines, unexpected residual inventory, incomplete fills, or accounting inconsistencies;
- dynamic order-rate pacing using the case-advertised API rate;
- competition-specific database and CSV files;
- heavy 30-tick research follow-ups disabled by default to prevent unnecessary API/I/O competition with the live path.

The preflight on competition day detected a 2-cent fee for both securities, maximum order sizes of 25,000 CRZY and 10,000 TAME, a 200-order/second advertised API rate, zero execution delay, zero starting positions, and zero open orders before arming live execution.

3 What was recorded

The research process was intentionally more data-intensive than the final class build. By the end of V6, the persistent research SQLite database contained 361 tender records, 10,712 follow-up snapshots, and 384,912 stored order-book-level rows. The versioned CSVs separately recorded live executions and individual hedge orders.

For competition day I intentionally disabled the large follow-up and book-level tables. This was not a limitation in capability; it was a deployment choice. The competition database still contains:

- 35 tender decisions;
- 8 execution sessions;
- 41 individual hedge-order events;
- 146 timestamped latency events;
- 520 position observations.

For each accepted tender I retained acceptance latency, booking latency, refreshed pre-hedge VWAP and edge, booking-period edge change, first/last order timing, burst duration, position-settlement timing, accounting-settlement timing, fill status, realized hedge VWAP, execution error, security-level realized P&L, fee-model error, fine delta, repair quantities, and rate-limit events. Thus the competition build omitted high-volume research history while retaining an audit trail sufficient to reconstruct the execution lifecycle.

This distinction is important for the repository. V1–V6 demonstrate the research capability; the competition build demonstrates the production choice to remove unnecessary work from the hot path.

4 Competition postmortem

4.1 Scope and outcome

The automated dataset covers seven complete scripted heats, 35 tenders total. After the scripted sequence ended, I participated in one additional manual round that was outside the engine’s logged system; I do not include that manual round in algorithm performance statistics. Within the seven scripted heats, five heats were profitable and two were flat because no tender passed the policy. There was no losing scripted heat.

The engine accepted 8 of 35 tenders (22.9%) and declined 27. All 8 accepted tenders realized positive P&L. Total realized security-level P&L was

$$\boxed{\$70,184.88}. \tag{5}$$

The best two scripted heats, which are the natural candidates for the assignment’s “best two cases” instruction, were:

Heat	Accepted tenders	Arrival-model P&L	Realized P&L
1	2	\$15,878.08	\$20,148.12
4	2	\$18,312.00	\$18,426.00

Heat 7 ranked third at \$12,033; Heat 2 ranked fourth at \$10,122.76; Heat 3 realized \$9,455; Heats 5 and 6 were \$0 because the engine found no qualifying offer.

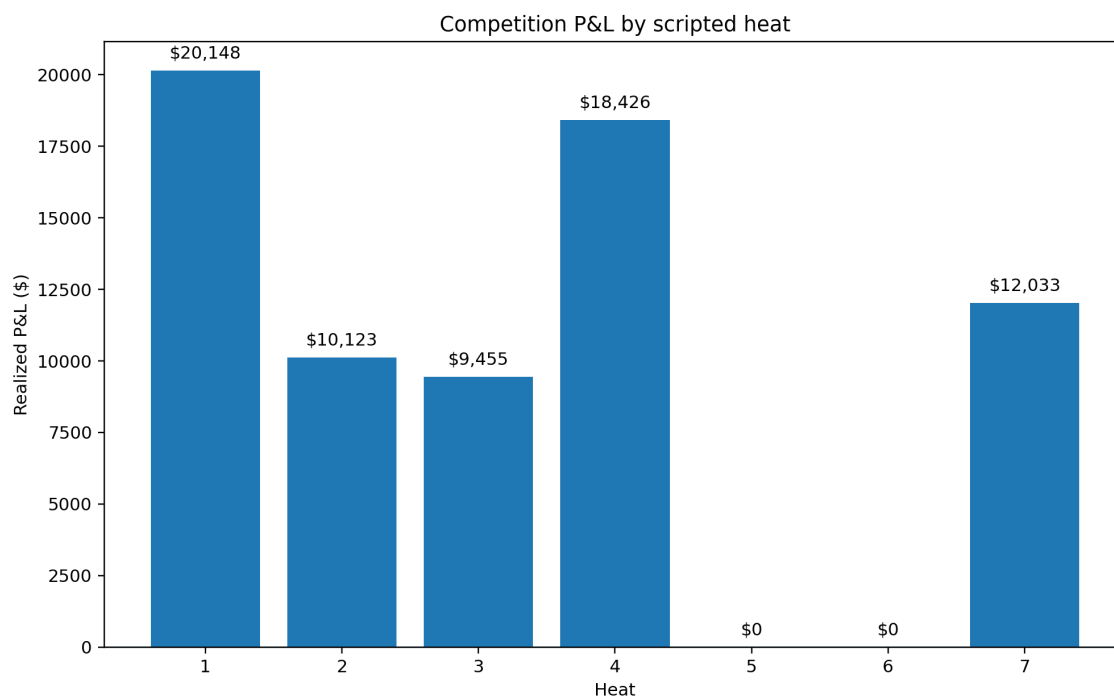


Figure 3: Realized P&L by scripted competition heat. Zero-P&L heats represent deliberate non-participation when no tender met the ex-ante rule.

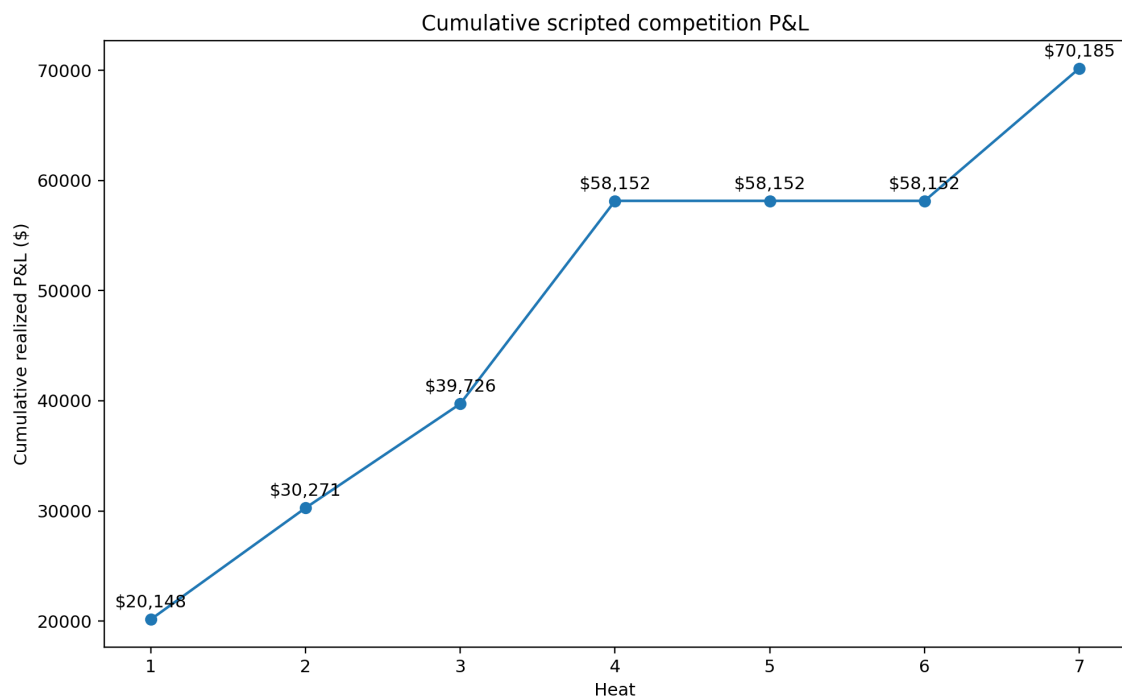


Figure 4: Cumulative realized P&L across the seven scripted heats.

4.2 The opportunity set was much worse than the demo samples

The lower class P&L relative to the best demo sessions was not an execution breakdown. It was primarily an opportunity-density change. V4.1 accepted 76 of 163 tenders (46.6%); the particularly rich V5 sample accepted 25 of 34 (73.5%); V6’s live V5 policy accepted 20 of 45 (44.4%). The competition produced only 8 qualifying tenders out of 35 (22.9%).

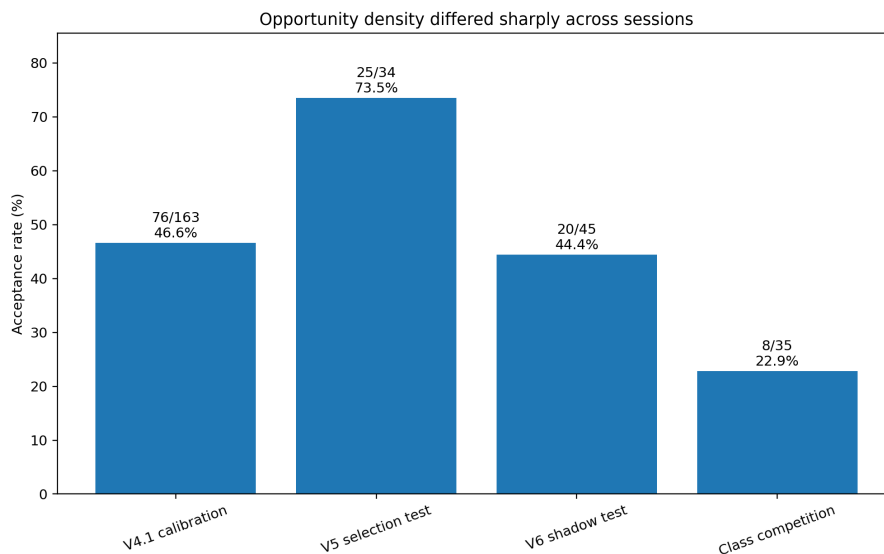


Figure 5: Acceptance rates across research/calibration sessions and the class competition. These rates are not performance scores; they show how different the underlying tender opportunity sets were.

In class, 21 of 35 tenders had a negative full-block after-fee edge at arrival, and 12 of 35 did not even have enough visible depth to liquidate the entire block immediately. Only nine offers had at least 5 cents/share of full-block after-fee edge; eight of those had full coverage and became the eight accepted trades.

That sparse opportunity set matters because the case only sends five tenders per five-minute heat. At roughly one decision per minute, a competent manual trader has time to inspect every tender. Automation therefore had less of a pure throughput advantage than it would have in a higher-arrival-rate environment. Its realized advantages were precision, full-depth calculation, consistent risk treatment, and rapid post-booking execution. If the same decision problem arrived at ten tenders per minute instead of one, the relative value of automated full-block valuation and execution would increase substantially; that is a conceptual scaling argument, not a claim tested by this particular case.

4.3 Why the full book mattered more than the leaderboard

The most important competition-day statistic is not final P&L. It is the difference between apparent top-of-book edge and actual block economics.

After subtracting the 2-cent market fee, 30 of 35 institutional tenders looked profitable if I used only the best bid or ask. Once I swept the quantity through the visible book, only 14 of 35 retained positive after-fee edge. There were therefore 16 false-positive “profitable” tenders created by top-of-book thinking alone.

For those 16 false positives, the average best-quote edge was approximately +16.19 cents/share, while the average full-block edge was approximately -15.99 cents/share. The sign of the trade could flip entirely because the required 65–75k shares consumed multiple levels of depth.

Among the 23 competition tenders that had enough visible depth to liquidate the entire block, blindly accepting all 23 would have produced contemporaneous after-fee liquidation economics of approximately

$$-\$75,623. \quad (6)$$

The eight tenders selected by the policy had observable arrival-time after-fee economics of

$$+\$57,119. \quad (7)$$

This is the cleanest evidence of selection edge in the project. It does not require me to know what happened later. The selected subset was economically different *before* subsequent random price movement was realized.

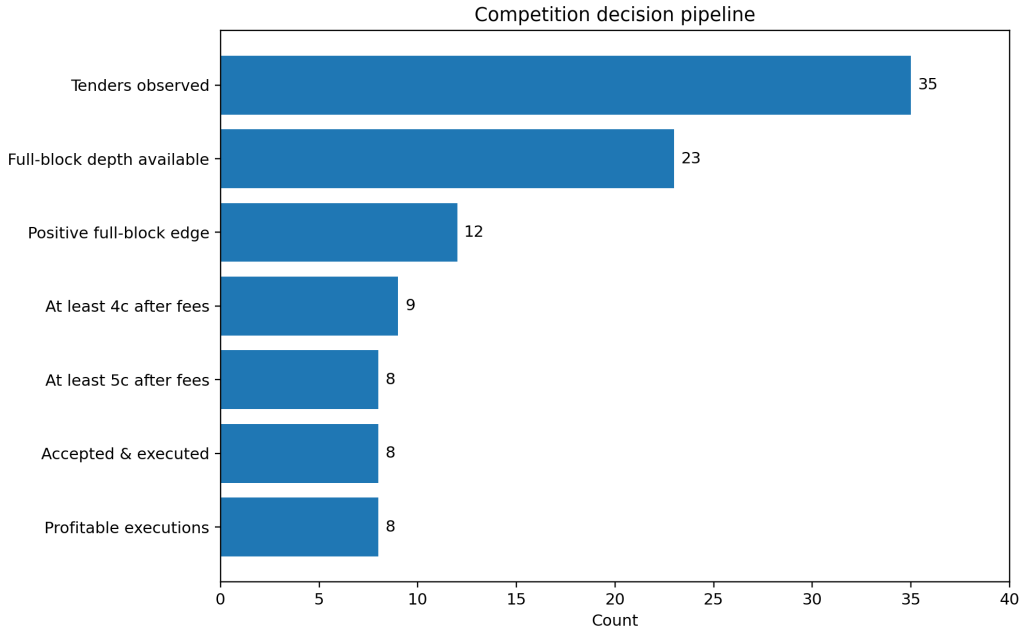


Figure 6: Nested competition decision pipeline. Full-depth coverage and full-block edge eliminated most of the apparently attractive tender stream before any live risk was taken.

4.4 Separate selection edge, booking randomness, and execution quality

The eight selected tenders had \$57,119.23 of predicted after-fee profit at their original arrival books. By the time RIT had booked the accepted inventory and I was permitted to hedge, their refreshed executable books implied approximately 69,827.88 of *after – fee profit*. *Final realized security P&L* was 70,184.88.

Thus the aggregate decomposition was approximately:

$$\text{Arrival-book edge} = \$57,119.23, \quad (8)$$

$$\text{Booking-period movement} = +\$12,708.65, \quad (9)$$

$$\text{Post-booking execution versus refreshed book} = +\$357.00, \quad (10)$$

$$\text{Realized P\&L} = \$70,184.88. \quad (11)$$

I consider this decomposition essential because it prevents me from claiming that the entire \$70,185 was deterministic “alpha.” Some of the realized improvement came from favorable random/liquidity movement during the unavoidable one-second booking interval. The strategy did not require that

favorable movement to have positive ex-ante economics, however: the selected blocks were already worth +\$57.1k on the arrival snapshot.

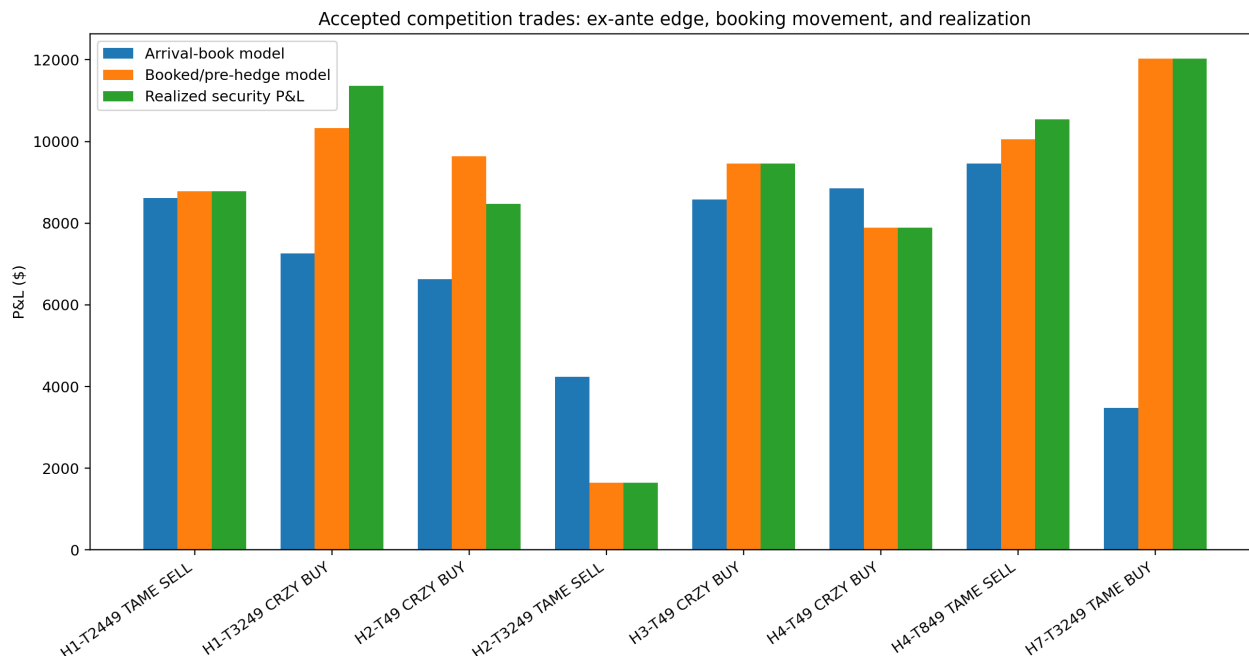


Figure 7: For each accepted competition tender: original arrival-book model, refreshed booked/pre-hedge model, and final realized security-level P&L.

The same figure shows why I do not treat every realized outcome as skill. The final Heat 7 TAME BUY arrived with only about 5.18 cents/share of after-fee edge and unfavorable full-depth imbalance near -0.227. During booking, its executable edge improved by approximately 12.78 cents/share and it ultimately realized \$12,033. A model that had been allowed to veto every apparently unfavorable liquidity state could easily have rejected one of the best realized trades. That was consistent with the V6 decision not to put a newly fitted probability model in control of core ≥ 5 -cent tenders.

The opposite case occurred in Heat 2. A TAME SELL arrived with approximately 6.33 cents/share of after-fee edge. During the booking interval the edge deteriorated by approximately 3.88 cents/share, leaving only about 2.46 cents/share before execution. It still realized +\$1,645. This is a concrete example of why I reserved 3 cents/share for short-horizon adverse movement rather than treating the arrival book as guaranteed.

4.5 Execution quality

Operationally, the final engine performed extremely well. Across the competition it unwound 553,000 accepted-tender shares through 41 hedge orders. All 41 orders were recorded as TRANSACTED. All eight accepted tenders ended at the target position. There were no repair orders, no API rate-limit hits, no new fines, and no execution failures.

Average accept-to-booked inventory latency was 1,003.1 ms. Once the booked liability was visible, the first hedge order began only about 2.0 ms later on average. The complete order-submission burst averaged 56.5 ms. The position returned to target approximately 259.6 ms after booking on average. The mandatory booking step, not Python computation, remained the dominant latency.

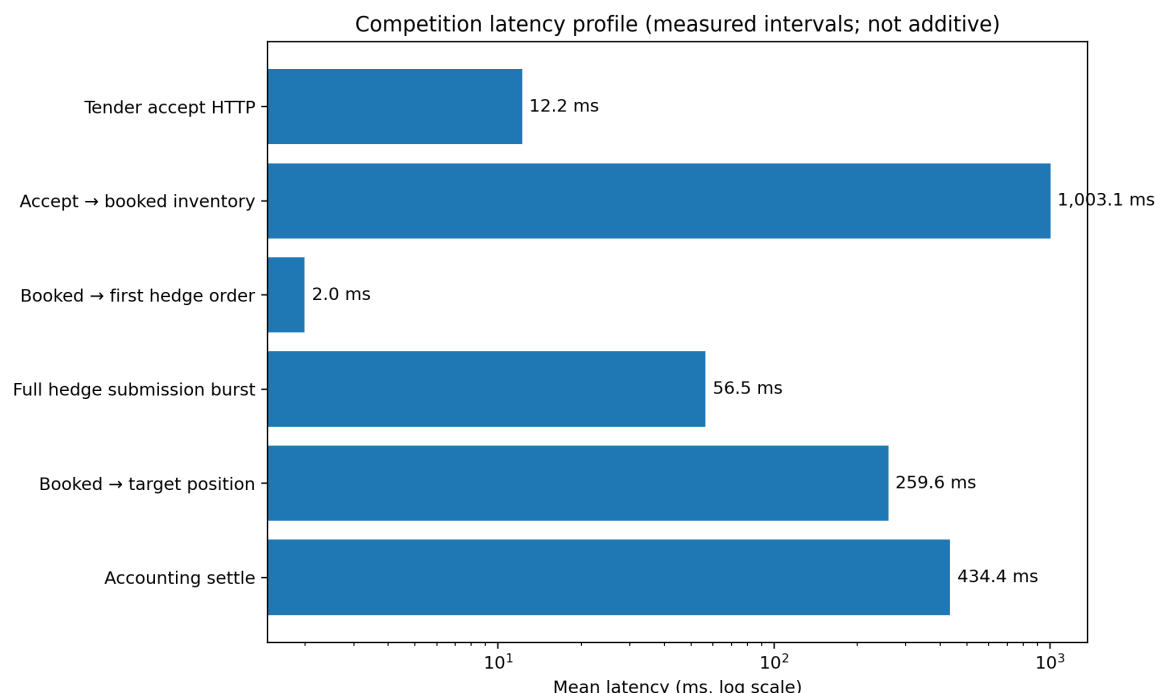


Figure 8: Mean competition latency intervals. The log axis makes the simulator’s roughly one-second booking synchronization floor visually obvious. Intervals overlap and should not be summed directly.

The operational result matters to my interpretation of the strategy. I was not leaving directional inventory open in order to manufacture leaderboard variance. The system accepted a client liability only when the current book offered a defensible spread, waited for the simulator to recognize that liability, and then neutralized it rapidly. That is closer to the risk-management mindset I associate with professional liability trading than maximizing a single five-minute P&L realization.

4.6 Independent validation from the official RIT report

The uploaded official RIT report corresponds to scripted Heat 2. It records my P&L as \$10,122.76, exactly matching the sum of the engine’s security-level realized P&L for the two accepted Heat 2 tenders. It also confirms zero fines, zero final positions, and the same accepted/declined tender sequence. This gives an independent check on the accounting logic in my database.

That report also provides class context for 32 traders. My Heat 2 P&L of \$10,122.76 was approximately 2.85 times the simulation average of \$3,545.72 and 1.76 times the median of \$5,761.91. The reported simulation range was -\$136,592.46 to +\$105,596.35. Heat 2 was only my fourth-best scripted heat, so I view this comparison as useful context rather than as the primary evidence for the strategy.

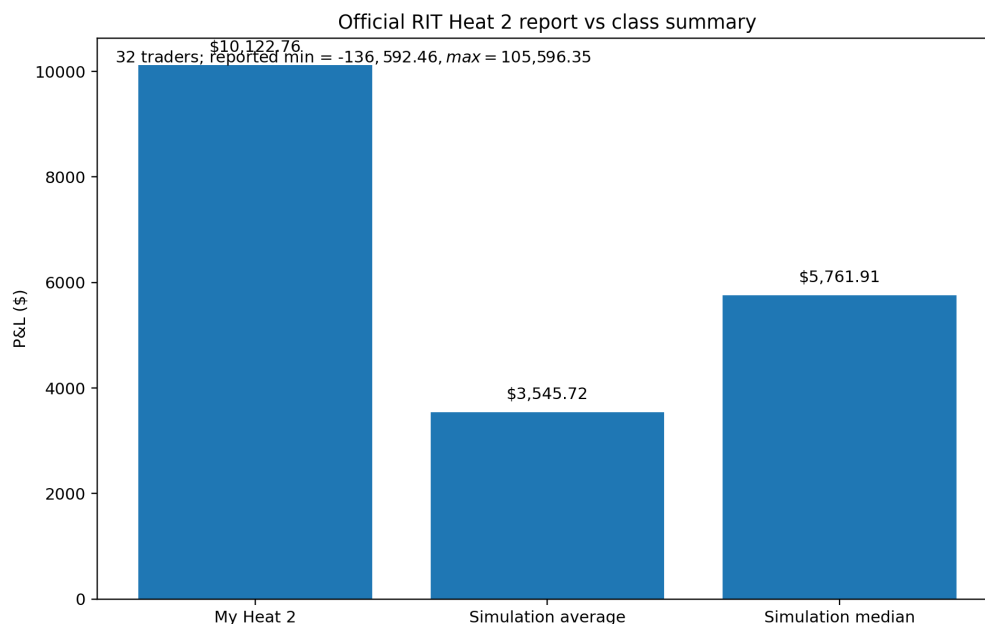


Figure 9: Official RIT Heat 2 result versus the simulation average and median reported in the same performance report.

5 Interpretation: edge, randomness, and the temptation to gamble

The LT3 case is useful precisely because it is possible to confuse a profitable realization with a good decision. The stated price process is a zero-mean random walk. If I accept a tender with negative current liquidation economics and then the market moves in my favor, I can make money; if it moves against me, I can lose substantially. A leaderboard over a small number of five-minute realizations does not, by itself, identify which outcome came from ex-ante edge and which came from exposure to randomness.

I therefore distinguish three categories:

1. **Observable selection edge:** the full-block spread available at the decision snapshot after fees and safety assumptions.
2. **Booking-period randomness/liquidity change:** the approximately one-second interval in which RIT had accepted the tender but had not yet booked the liability, preventing a safe hedge.
3. **Execution quality:** the difference between the refreshed pre-hedge book and the actual market-order fills.

My main evidence lies in category 1. The selected tenders had +\$57.1k of observable arrival-book economics, while the fully liquidatable opportunity pool as a whole had -\$75.6k. Category 2 happened to add another +\$12.7k in class. Category 3 added only about +\$357, which is exactly what I would expect from an execution engine whose purpose is to transfer the modeled book into actual fills with little additional risk.

I cannot infer the exact strategies used by other students from their final P&L. A higher score could come from better passive execution, different assumptions about liquidity replenishment, accepting more random-walk exposure, or some combination. I therefore would not write that a specific classmate “gambled” without their order history. What I can say is that any strategy accepting a tender whose current full-block liquidation edge is negative necessarily depends on something beyond the contemporaneously observable block spread: later price movement, later liquidity, passive fills, or another forecast.

Put less diplomatically, the *ignoramus* interpretation of LT3 is to equate a favorable best quote with a profitable 65–75k-share block. The class data falsified that shortcut directly: 30 of 35 tenders looked

profitable at the best quote after fees, but only 14 of 35 remained positive on a full-block sweep.¹

I was therefore not disappointed by declining tenders that later happened to work. The process was intentionally asymmetric: I wanted to be paid for liquidity provision before taking the liability, not to turn an institutional block into a directional bet after the fact. In a larger sample, process consistency and risk-adjusted expectancy matter more to me than whether the most aggressive trader wins one five-minute realization.

6 What the project established

The project left me with several conclusions that I would retain even if I rewrote the final Canvas submission much more briefly.

1. **Top-of-book spread is not block edge.** The class data generated 16 false positives where the best quote looked profitable but the full block was not.
2. **Liquidity selection was economically material.** Fully covered tenders as a group had -\$75.6k of immediate after-fee economics; the selected subset had +\$57.1k before later market movement.
3. **The dominant execution constraint was simulator booking latency.** Research identified an approximately 0.95–1.00 second liability-booking synchronization floor. Once booked, the hedge started within a few milliseconds.
4. **The V4 failure was informative.** Removing the booking gate made the engine faster but triggered large front-running/speculation fines, proving that an HTTP accept response and a booked liability were not equivalent simulator states.
5. **Full-depth imbalance contained a repeatable liquidity-stability signal.** The effect survived independent research samples, but a more complicated V6 model did not improve fresh economic decisions enough to justify deployment.
6. **Simple rules can dominate a more sophisticated model when data are limited.** V6 was statistically predictive but rejected a profitable marginal trade that V5 correctly accepted.
7. **The class opportunity set was unusually sparse relative to several demo runs.** Competition acceptance was only 22.9%, limiting total P&L despite flawless execution.
8. **Operational risk control mattered.** The final class build executed 41/41 orders, ended flat after every accepted tender, and generated zero fines, repairs, and rate-limit hits.

7 Limitations and what I would test next

I would not overstate the evidence. The competition itself contains only 35 tenders and eight executed trades. The 100% hit rate on accepted tenders is therefore descriptive, not a guarantee. The competition build also intentionally disabled 30-tick follow-ups for declined tenders, so the class session by itself cannot provide an unbiased re-test of the imbalance hypothesis; that inference comes from the larger pre-competition research dataset.

The simulator is also not a literal model of institutional market microstructure. The roughly one-second booking delay is a property of the RIT environment. Fees, maximum order sizes, and order-book mechanics are case parameters. Real markets introduce queue priority, hidden liquidity, venue fragmentation, market impact, information leakage, asynchronous acknowledgments, and much more complex execution choices.

If I continued the research rather than submitting the assignment, the most interesting next tests would be:

- increase tender arrival intensity to measure the throughput advantage of automation;

¹This sentence is intentionally personal in tone and can be deleted from the formal Canvas submission without changing the argument.

- use a larger out-of-sample dataset to estimate the conditional distribution of booking-period edge deterioration rather than only a mean or binary survival probability;
- test passive or marketable-limit execution only after preserving a clean aggressive baseline;
- separate depth persistence from exogenous price movement more explicitly;
- perform a formal walk-forward comparison of the deterministic V5 policy against any future model, with promotion rules fixed before the test.

The V6 result makes me especially cautious about further feature engineering. The correct stopping point for this assignment was not the most complicated model I could build; it was the simplest strategy that survived fresh tests and could be executed reliably under the case constraints.

8 Concluding Remarks (What I'd Have Written in Canvas to Address the Prompt)

ChatGPT first tried vehemently to have me rewrite a lot of my initial takeaways nitpicking the simulation itself in regard to realism. After having some difficulty and carrying my beliefs through the report, it sanitized my criticisms of Rotman and the exercise itself in favor of answering your actual question, which I agree was the best call. However, I wanted to add my unfiltered opinion somewhere, and after the conversation I had with you after class on the 22nd, I feel comfortable doing so. Mainly because you came off as very reasonable, and I know you're a real practitioner who's likely heard worse on the trading floor.

The competition disappointed me. I'm not against "manual," aka discretionary, trading at all. I personally have a good 10k hours in front of the screen (futures and options mostly), and I think you really nailed the most important aspect with your reminders, even during the competition, to think about how you *feel*. You can't always know what's broken in the moment, nor can you know every single individual statistic that's causing a certain result in a moment either; you can stop and think about how you feel and how that's affecting your cognition. I think that a lot of contemporary aspiring quants have written off the importance of mentality, and will sadly never benefit from reading *Trading in the Zone* or knowing about trading coaches, journaling, etc. I personally believe that professionals can and should do that, but unlike retail traders it shouldn't be analytical, since we know how to assess performance quantitatively.

Still, I also believe that even assessing the strategy has its limitations. I'm excited for you to see my final project because it's a real start-to-finish ES L2 MBO Depth 10 pipeline that uses GEX/DEX and Open Interest from the options chain to go from chaotic tick data to feature engineering 1k+ metrics and using LGBM/SHAP and a bunch of other models to eventually define a shock and trade it algorithmically. The coolest part is I have actual performance in real life of said algo (it's scalping, so I call it Apache in homage to the Native Americans). I will also show another dealing with TWAP and VWAP mean reversion that I learned from a hedge fund I spent time at; that's called TREmor (trend mean reversion).

I digress. The important takeaway as to why I trashed the exercise and felt like a lot of my classmates were gambling was because of the **underlying market**. I still believe that my algo in our project here was the most institutional-grade, in that I didn't lose any capital (aside from the final bonus round where I turned it off). I demonstrated how, according to the model I used here, the losses would've been almost as much as my profit. That's the kind of mindset I believe a professional has. If I wanted to chase wins, I would've. No offense to my friend in the class, but even today he was hypothesizing his orders had outsized impact on the book and thus everyone else. That's false. To really have an argument like that, I'd want to hear about him or anyone (not singling him out) discovering the artificial 950-970 ms latency boundary the simulation imposed independently. If you're that smart, you're above

the architecture like that.

All that to say, I kind of still feel like the ones who treated the exercise with the respect you need to have in a real market and at a real fund were the ones who the exercise wasn't suited for. If it was 10 tender offers a minute, where a human couldn't physically handle the trading and speed really mattered, I think I would've won. (I didn't include them here, but I did simulate that scenario after seeing how the market in our competition operated live versus the practice market.)

The best strategy for me, looking back, might've been to just know that algorithms were disadvantaged in the exercise and, like my classmates, practice trading manually if winning was the only goal. I didn't want to put a target on my back too early, and I still don't want to, but I'm going to link my GitHub, so it'll get out sooner or later; I have some trading experience I'm not being totally upfront about.

Overall: Execution and risk control were handled by the final competition engine rather than manually. After accepting a tender, the system waited for RIT to book the resulting liability before initiating the hedge, then pipelined the unwind in legal maximum-size market-order slices. Across the seven scripted heats, all 41 hedge orders were transacted, every accepted tender was fully flattened, and the engine incurred zero fines, repair orders, rate-limit events, or execution failures (see Section 4.5 of the attached report).

The two best scripted cases were Heat 1 and Heat 4. Their relative performance is shown earlier in Figure 3, while Figure 4 shows how those gains accumulated across the full scripted competition sequence. Figure 7 is especially useful for interpreting the trades themselves because it separates the original arrival-book edge from the booked/pre-hedge estimate and the final realized P&L, rather than presenting the realized result alone.

8.1 Heat 1 (+\$20,148.12)

Heat 1 was the best scripted case. The engine accepted two of the five institutional tenders: a 75,000-share TAME SELL and a 73,000-share CRZY BUY. Their combined arrival-book model predicted approximately \$15,878.08 of after-fee liquidation profit, while the two trades ultimately realized \$20,148.12. Both positions were completely unwound with no fines, residual inventory, or repair orders.

This heat also illustrates the distinction made throughout the report between ex-ante edge and favorable subsequent movement. The strategy did not need the later improvement in prices to justify taking either tender; both had already passed the full-depth, fee, safety-buffer, and risk constraints at the time of acceptance. Figure 7 visualizes that progression from arrival-book value to pre-hedge value to final realization.

8.2 Heat 4 (+\$18,426.00)

Heat 4 was the second-best scripted case. The engine accepted a 70,000-share CRZY BUY and a 67,000-share TAME SELL. The arrival-book estimates totaled approximately \$18,312.00, and realized P&L was \$18,426.00, meaning the final outcome stayed remarkably close to what the strategy could justify from the contemporaneous order book.

Of the two best heats, Heat 4 is arguably the cleaner demonstration of the underlying thesis because almost all of the realized result was already visible in the ex-ante full-block economics rather than being created by favorable booking-period movement. Together, Heat 1 and Heat 4 summarize what I wanted the system to do: identify genuinely positive block-level liquidity opportunities, accept only those opportunities, and remove the resulting inventory quickly and mechanically rather than convert the tender into a directional bet.

9 Reproducibility and repository map

The repository history itself is part of the explanation: **Repository:** github.com/Nathaniel-Coulter/Columbia-Rot

File	Purpose
lt3_edge_engine.py	V1 full-block VWAP, fee/buffer/risk rule, basic execution.
lt3_edge_engine_v2.py	Persistent SQLite research logger, 30-tick follow-ups, full-depth book capture.
lt3_edge_engine_v3.py	First live execution calibration, per-slice timing/fill/P&L telemetry.
lt3_edge_engine_v4.py	Immediate post-accept pipelining; useful failed experiment that exposed the booking/fine boundary.
lt3_edge_engine_v4_1.py	Correct tender-booking gate plus pipelined hedge; large validation sample.
lt3_edge_engine_v5.py	Final deterministic alpha logic: 5-cent core plus bounded 4–5 cent deep-imbalance regime.
V6 logs / model tables	Frozen booking-risk shadow model; not promoted after fresh economic comparison.
lt3_competition_engine.py	Hardened V5 deployment with preflight, fail-closed checks, lock, separate logs, and reduced research overhead.
lt3_training.sqlite + <i>versioned copies</i>	Research database containing tender, follow-up, book-depth, execution, and model-calibration history.
lt3_competition.sqlite	Final class audit database containing tender decisions, executions, latency events, order events, and position observations.

A Research/version summary

Stage	Tenders	Executed	Orders	Shares	Main purpose/result
V2	76	40 paper accepts	–	–	Persistent paper/research logger.
V3	30	12	55	852,000	First live execution calibration; \$111,324 tender/hedge economics.
V4	13	7	34	486,000	Immediate pipeline; execution faster but simulator fines invalidated architecture.
V4.1	163	76	372	5,327,000	Booking gate + pipeline; \$709,829 realized, 76/76 profitable, zero fines.
V5	34	25	121	1,738,000	5-cent core threshold + marginal imbalance regime; \$187,118 realized.
V6	45	20	100	1,382,000	V5 remained live while booking-risk model ran in shadow; \$189,287 realized.
Competition	35	8	41	553,000	Hardened final engine; \$70,184.88 realized, zero fines.

P&L totals across research versions are not directly comparable because the number and quality of heats differed substantially. Their role here is to document calibration and engineering progression, not to rank versions by raw cumulative dollars.

B Competition trade table

Heat / Tender	Instrument / Side	Arrival model	Pre-hedge model	Realized
H1 / 2449	TAME SELL	\$8,619.88	\$8,787.24	\$8,787.24
H1 / 3249	CRZY BUY	\$7,258.20	\$10,326.64	\$11,360.88
H2 / 49	CRZY BUY	\$6,631.00	\$9,635.00	\$8,477.76
H2 / 3249	TAME SELL	\$4,243.00	\$1,645.00	\$1,645.00
H3 / 49	CRZY BUY	\$8,583.00	\$9,455.00	\$9,455.00
H4 / 49	CRZY BUY	\$8,856.00	\$7,888.00	\$7,888.00
H4 / 849	TAME SELL	\$9,456.00	\$10,058.00	\$10,538.00
H7 / 3249	TAME BUY	\$3,472.15	\$12,033.00	\$12,033.00
Total		\$57,119.23	\$69,827.88	\$70,184.88

C Sources

1. Mak, Kevin and Tom McCurdy. *RIT Case Brief – LT3: Liability Trading 3*, Rotman Interactive Trader, Build 1.08.
2. Rotman Interactive Trader. *RIT Performance Report – Liability Trading 3 Case*, trader nhc2123, Heat 2, September 22, 2026.
3. Coulter, Nathaniel H. *LT3 Research and Execution Repository*, 2026.
github.com/Nathaniel-Coulter/Columbia-Rotman-Trading-Competition.
4. Local research artifacts: `lt3_training.sqlite`, versioned tender/execution/order CSVs, `lt3_competition.sqlite`, Terminal outputs `9_22_2026.txt`.